

Platform blueprint

v1.0

Idea-as-a-Service

An AI-native software factory that takes a product idea (ERP, web, mobile, IoT, network, cyber) from planning to deployment and maintenance, using local LLMs, type-safe AI, an AI harness and agents, under a proper SDLC.

Contents: vision and scope, architecture (7 diagrams), SDLC with gates, type-safe AI, local models, hardware and software requirements, ERP worked example, maintenance, security and compliance, team and cost, roadmap, risks, 30-day checklist.

All costs, throughputs and timelines are planning estimates (2026 prices) to be validated by benchmarking. Model names date quickly; the architecture does not depend on any single model.

1. Executive summary

You asked for an "IaaS" that builds products for people who have no engineering team. I have read this as **Idea-as-a-Service**: a client brings an idea and receives a running, maintained product. If you meant Infrastructure-as-a-Service, sections 13-14 (hardware, software) and the deployment diagram still apply.

Core design decision. Do not build "one big agent". Build a **deterministic control plane** (workflow engine, policy gates, typed artifact registry) that *employs* role-specialised agents, each wrapped in an **AI harness** that forces typed outputs, runs code only inside sandboxes, and accepts work only when independent verifiers (compilers, type-checkers, tests, scanners) pass. Local open-weight models supply the intelligence; humans sign off at a small number of high-risk gates.

What the platform delivers

- **Planning:** an interviewing agent turns an idea into a PRD, acceptance criteria, a fixed-scope quote and a risk register.
- **Development:** architecture and typed contracts first, then tests, then code, in small reviewed pull requests built inside sandboxes.
- **Deployment:** infrastructure-as-code, signed images, SBOM, canary release, one-click rollback, into the client's cloud or yours.
- **Maintenance:** monitoring, ticket triage, ticket-to-patch loop, dependency and security patching, monthly releases, SLOs.
- **Domains:** web/ERP/SaaS first; then mobile, cyber-security products, network automation, IoT/embedded, each with its own tool-and-test squad.

Honest expectations

- Agents reliably produce **CRUD-heavy business software on mature frameworks**. Novel algorithms, deep hardware timing, regulated logic (tax, payroll, medical) still need human experts. The design makes those expert touchpoints explicit and cheap instead of pretending they vanish.
- Autonomy is a **dial per project and per domain**. Start supervised (human approves every merge); raise autonomy only where your own eval results prove it (e.g. 90%+ of golden tasks pass unattended).
- For ERP, **start from an open-source core** (ERPNext or Odoo Community) and customise, instead of generating accounting logic from scratch.
- Cyber and network work touching live systems is **authorised, scoped and gated**; offensive tooling runs only in an isolated range against targets the client owns.

Recommended path

Rent an 8-GPU node for 4-8 weeks, benchmark open models on your own task set, build the harness + web/ERP squad, ship one pilot ERP with human gates, then expand domain by domain (roadmap in section 18).

2. Scope, personas and principles

Persona	Need	What they get
Solo founder / SME owner	Product without hiring	Idea to production, fixed quote, monthly plan
Domain expert (clinic, factory, school)	Software that fits their process	Interview-driven ERP/CRM/portal, Indian/local compliance packs

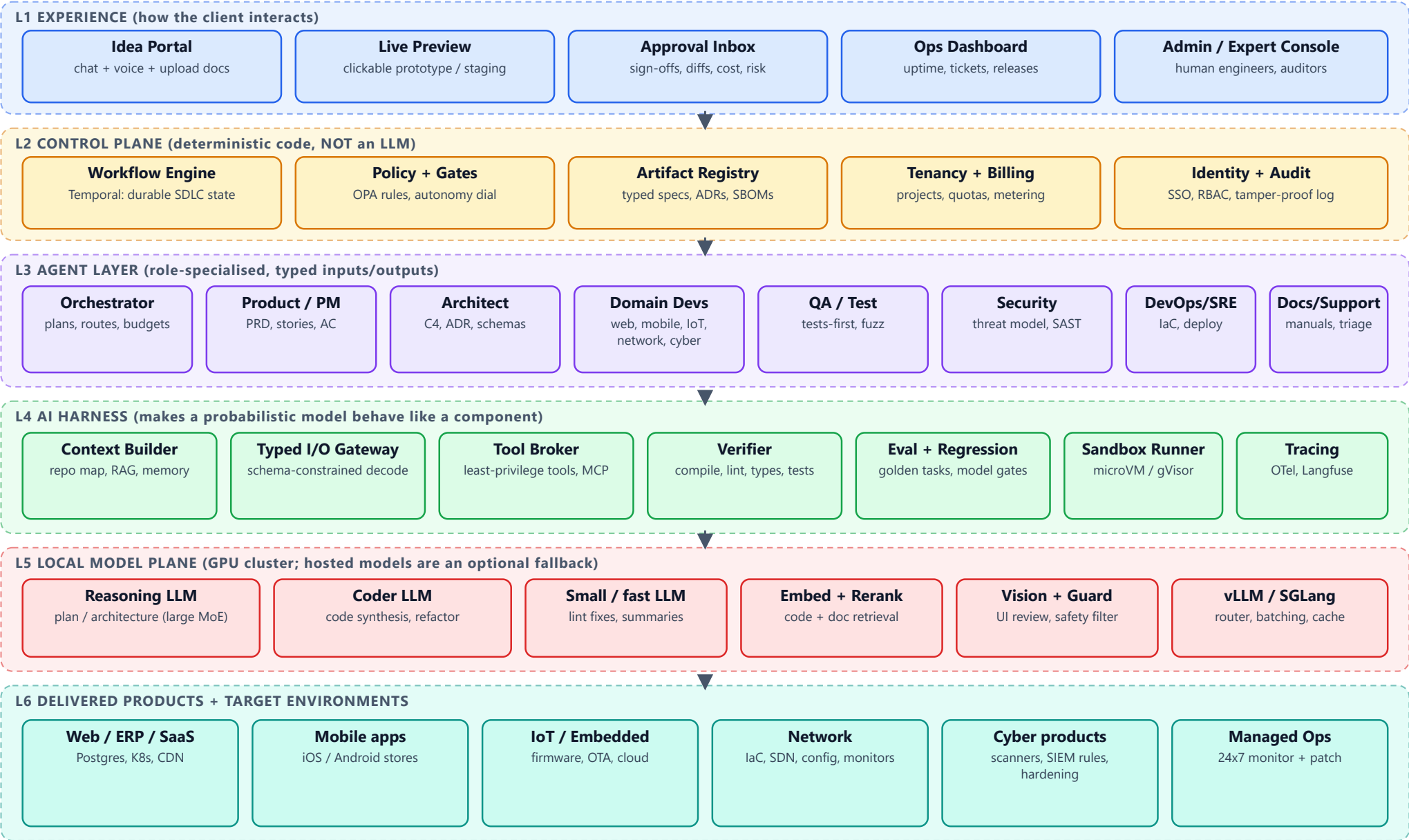
Persona	Need	What they get
Small IT team	Force multiplier	Same platform in "assist" mode on their repo
You (platform operator)	Margin and quality	Per-project cost/quality metrics, expert-review marketplace

Design principles

- **Contracts before code.** Every hand-off is a typed artifact (JSON-schema, OpenAPI, protobuf, SQL DDL, IaC), so agents cannot drift.
- **The model proposes, deterministic code disposes.** No LLM approves its own work; compilers, tests and policy engines do.
- **Tests first.** Acceptance criteria become executable tests before implementation starts.
- **Least privilege + isolation.** Generated code and agent tools run in microVM sandboxes with no default network egress.
- **Local first.** Client data and code never leave your infrastructure unless the client opts in to a hosted model.
- **Everything is traced and replayable.** Every prompt, tool call and verdict is logged; every failure becomes a regression eval.
- **Client owns the output.** Code lives in the client's repo, standard stack, no lock-in to your platform to keep running.

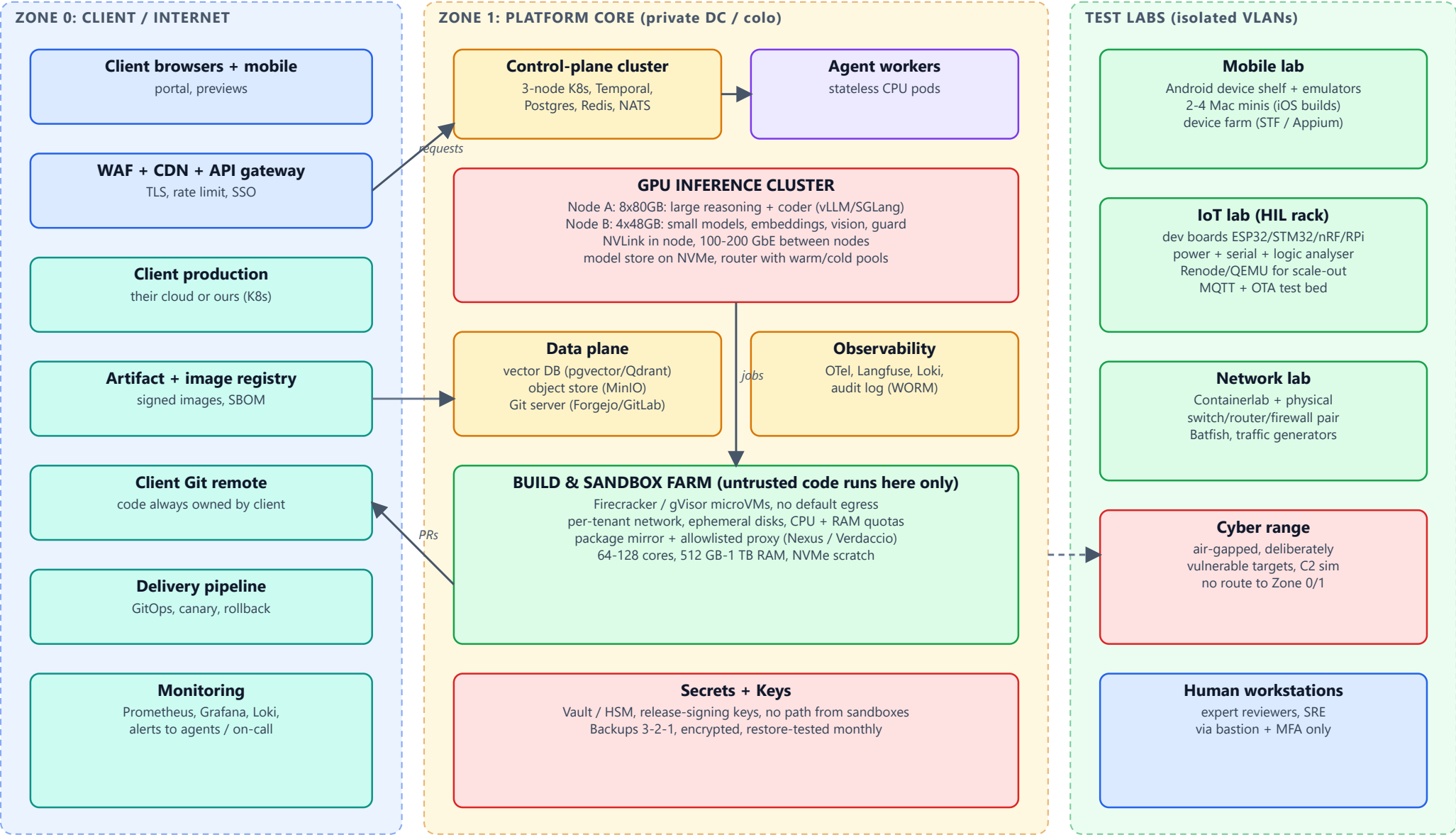
3. Architecture diagram 1: layered platform

Six layers. The control plane (L2) and harness (L4) are ordinary deterministic software; only L3 agents and L5 models are probabilistic.



4. Architecture diagram 2: deployment topology and trust zones

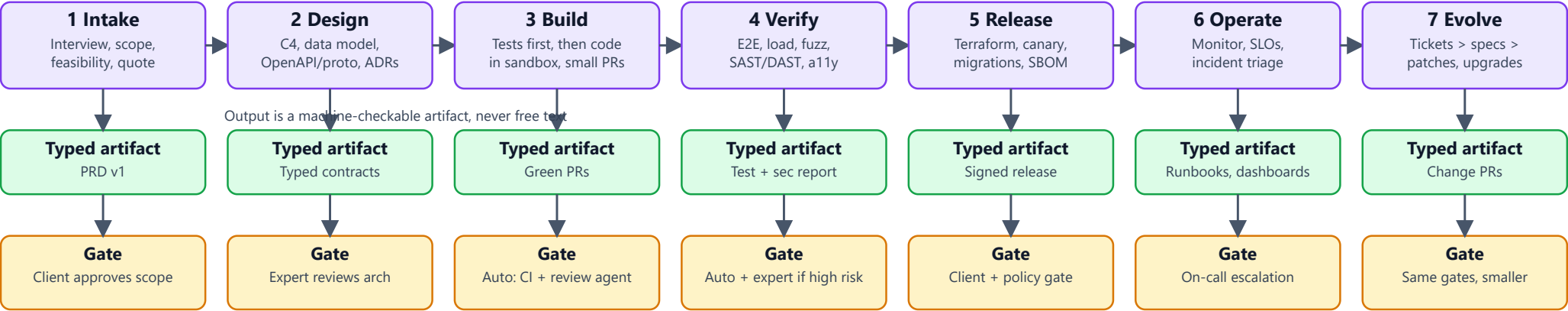
Untrusted generated code runs only in the sandbox farm; secrets and signing keys have no route from it. Test labs are isolated VLANs.



5. Architecture diagram 3: lifecycle pipeline (SDLC)

Seven stages; each ends in a machine-checkable artifact and a gate. Maintenance feeds back into design.

Stages (agents do the work)



Failed gate: auto-retry (max N) then escalate to a human. Anything above a small patch in maintenance returns to stage 1/2.



6. SDLC in detail: planning to maintenance

Stage	Agents	Key activities	Typed artifacts	Gate (who approves)
1 Intake / plan	Product, Orchestrator	Socratic interview, competitor scan, feasibility, scope + quote, risk register	PRD, user stories, acceptance criteria (Gherkin), estimate	Client approves scope and price
2 Design	Architect, Security	C4 diagrams, data model (3NF), API contracts, ADRs, STRIDE threat model, stack choice	OpenAPI/protobuf, SQL DDL, IaC skeleton, threat model	Expert reviews (mandatory for finance, health, IoT safety)
3 Build	Domain devs, QA	Tests from acceptance criteria, then implementation in sandbox; small PRs; review agent + static analysis	Green PRs, generated test suite, migration scripts	Automatic: CI + reviewer agent + policy
4 Verify	QA, Security	Integration/E2E, load, property + fuzz, SAST/DAST, dependency + licence scan, accessibility, mutation tests	Test report, coverage, security report, SBOM	Auto; expert for high-risk changes; client UAT
5 Release	Release/Ops	Terraform, migrations, canary + auto-rollback, signed images, docs and training material	Signed release, runbook, rollback plan	Client + policy approve go-live
6 Operate	SRE, Support	SLO monitoring, alert triage, incident response, backup + restore drills, capacity	Dashboards, incident reports, postmortems	On-call human escalation for P1
7 Evolve	Product, all	Ticket to spec to patch; dependency updates; feature requests re-enter stage 1/2 by size	Change PRs, release notes	Same gates, scaled by risk

Human sign-off points (six)

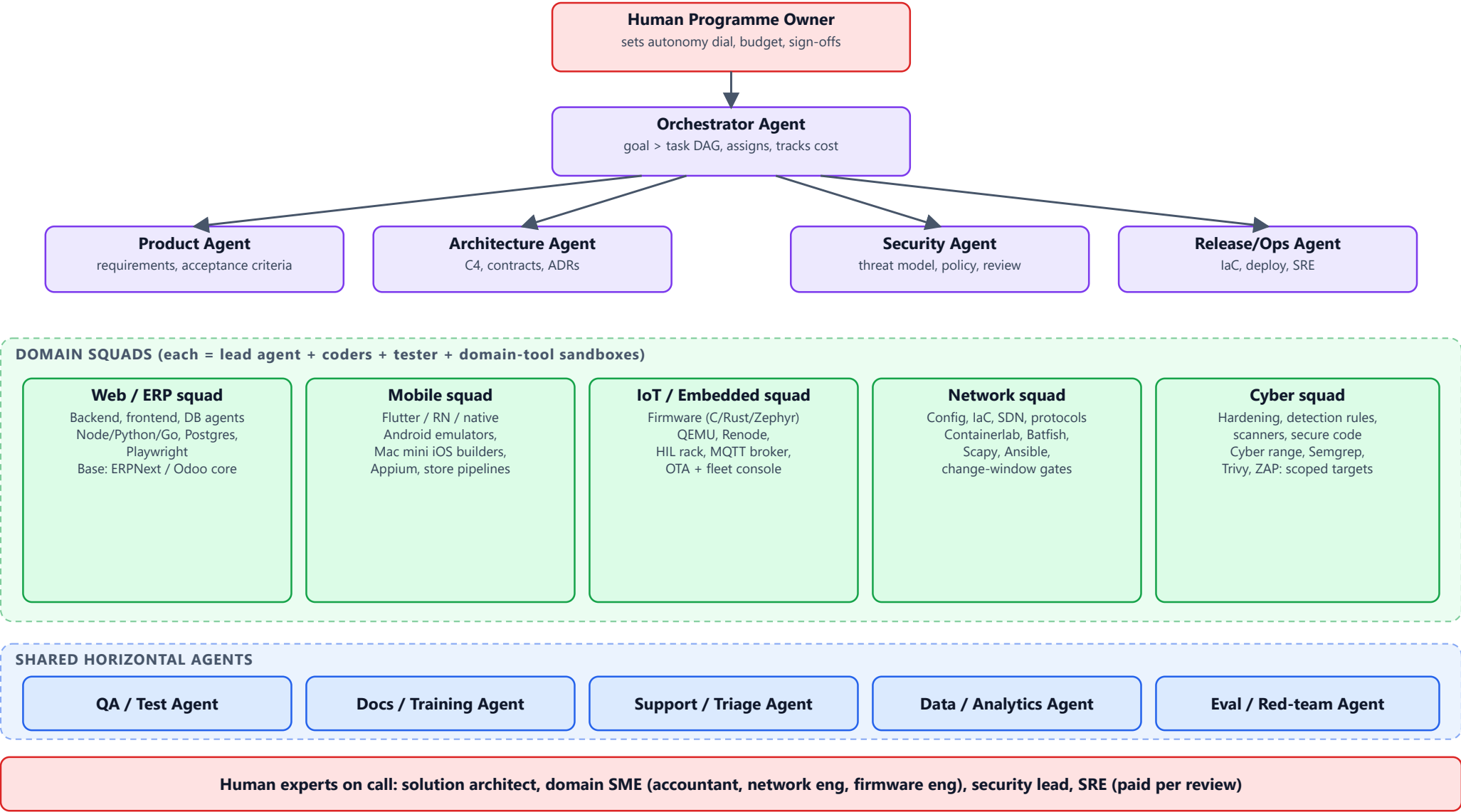
(1) scope and price, (2) architecture for high-risk domains, (3) regulated logic (tax, payroll, safety-critical firmware), (4) production release, (5) any destructive or security-sensitive action on live systems, (6) model or prompt changes that fail regression evals. Everything else is automated with supervision.

Autonomy dial

Level	Behaviour	Use when
L0 Assist	Agents draft; human does merges and releases	New domain, first pilot
L1 Supervised	Agents merge to branch; human approves each release	Default for production
L2 Bounded auto	Agents merge and release low-risk changes (patches, deps) automatically	90%+ eval pass rate for that change class
L3 Autonomous ops	Auto-remediation runbooks within blast-radius limits	Mature web/ERP squad only

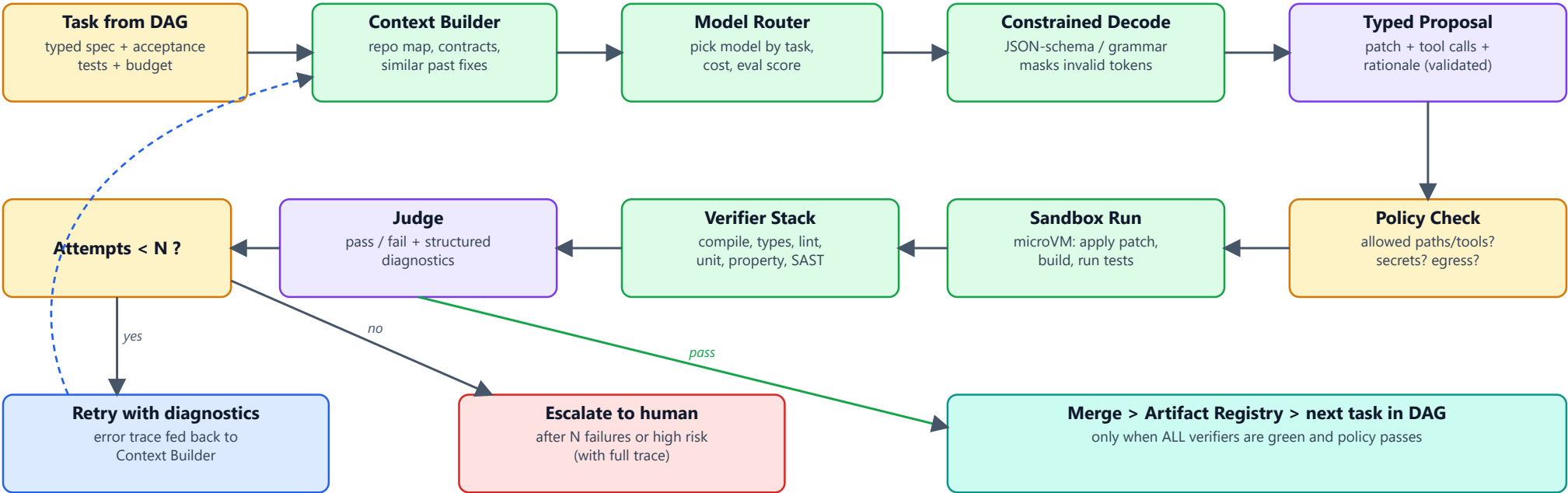
7. Architecture diagram 4: agent organisation

Orchestrator + programme agents, one squad per domain with its own toolchain and sandboxes, shared horizontal agents, and paid human experts.



8. Architecture diagram 5: the harness loop for one task

How a single unit of work is executed, verified and either merged, retried with diagnostics, or escalated.



Every step is traced (prompt, model, tokens, tool calls, verdict), so failures become new eval cases.

Key idea: the model proposes; deterministic code disposes. The model never judges its own output alone.

9. Type-safe AI: six layers of defence

"Type-safe" here means an LLM output is never trusted as free text. Each layer removes a class of failure; together they push error rates down to what tests and reviewers can catch.

Layer	Mechanism	Tools (recommended / alternatives)	Failure it prevents
1 Constrained decoding	Compile JSON-schema / grammar to a token mask so invalid output is impossible	XGrammar, Outlines, Ilguidance built into vLLM/SGLang; llama.cpp GBNF	Malformed JSON, wrong enum, missing fields
2 Schema-first agent I/O	Every agent input/output is a versioned schema; validated at the gateway	Pydantic v2 / Zod, JSON Schema, BAML, Instructor	Contract drift between agents
3 Contract-first product code	OpenAPI / protobuf / SQL DDL produced by Architect, code generated against it	openapi-generator, buf, Prisma/SQLAlchemy, tRPC	API mismatch between frontend, backend, mobile
4 Typed target languages	Prefer TypeScript strict, Rust, Go, Python + mypy/pyright strict; type-check in the loop	tsc, mypy/pyright, cargo check, clippy	Whole class of runtime errors
5 Syntactic + structural gates	Tree-sitter parse, lint, formatter, forbidden-API rules	Tree-sitter, ESLint, Ruff, Semgrep custom rules	Broken syntax, banned patterns, secrets in code
6 Behavioural verification	Tests written first, plus property tests, mutation tests, golden datasets	pytest, Jest, Hypothesis/fast-check, Stryker/mutmut, Playwright	Logic errors types cannot see

Limit: types and schemas guarantee *shape*, not *truth*. A perfectly typed tax calculation can still be wrong. That is why acceptance tests, property tests and expert sign-off exist for business-critical logic.

10. AI harness and local model plane

Harness components

Component	Responsibility	Implementation options
Context builder	Repo map, symbol index, contracts, similar past fixes, project memory; keeps prompts small	Tree-sitter + ctags, pgvector/Qdrant, reranker, prefix caching
Model router	Chooses model per task by capability, cost, latency and current eval score; falls back on failure	LiteLLM / custom router over vLLM + SGLang endpoints
Typed I/O gateway	Constrained decoding + schema validation + retries	XGrammar/Outlines, Pydantic/Zod
Tool broker	Exposes only the tools a role needs (git, shell, browser, DB, cloud CLIs) with argument schemas and rate limits; MCP-compatible	MCP servers, OPA policy, custom broker
Sandbox runner	Runs build/test/commands in ephemeral isolation, no egress by default	Firecracker, gVisor, Kata, Docker rootless; Tetragon/Falco audit
Verifier stack	Independent deterministic checks; the only authority for "done"	CI runners, linters, test frameworks, SAST/DAST
Eval + regression	Golden tasks per domain; nightly and per-model-change runs; blocks rollout on regression	Custom harness, promptfoo, SWE-bench-style private set

Component	Responsibility	Implementation options
Tracing + cost	Full trace per task, token and GPU cost per project	OpenTelemetry, Langfuse, OpenLLMetry
Durable workflow	Long-running, resumable, human-in-the-loop SDLC state machine	Temporal (recommended), Argo Workflows, Prefect

Local model plane (route by task; re-benchmark quarterly)

Role	Model class (examples as of 2026)	Serving footprint	Used by
Large reasoning / planning	Open large MoE: DeepSeek-V3/R1 family, Qwen3 large, GLM, Kimi (FP8/INT4)	8x80GB GPUs (or 4x with INT4); slow, expensive, used sparingly	Product, Architect, Security, hard debugging
Coder (workhorse)	Qwen3-Coder / Qwen2.5-Coder 32B, DeepSeek-Coder-V2, Codestral class	1-2x 80GB GPUs per replica; many concurrent requests	Domain devs, QA, DevOps
Small / fast	7-14B coder or general (Qwen, Phi, Llama, Gemma)	1 GPU shared	Lint fixes, summaries, commit messages, routing
Embeddings + reranker	bge / e5 / Qwen embedding + reranker models	Fraction of a GPU	Code and doc retrieval
Vision	Open VLM (Qwen-VL, InternVL, Llama vision)	1 GPU	UI screenshot review, diagram/wireframe ingestion
Guard	Llama Guard / ShieldGemma class + rule filters	Fraction of a GPU	Prompt-injection and unsafe-action screening
Optional hosted fallback	Frontier API models, opt-in per client, redacted context	None locally	Only when local models fail an eval and client consents

Serving: vLLM or SGLang with continuous batching, paged attention, prefix caching (agents share long system prompts), tensor parallelism inside a node, speculative decoding where useful. Quantise (FP8/AWQ/GPTQ) large models; check quality on *your* evals before adopting any quantisation. Fine-tuning (LoRA) on accepted PRs and your house frameworks is a Phase 3+ optimisation, not a day-one need.

11. Domain squads: what changes per product type

Domain	Typical stack generated	Verification environment	Extra gates / cautions
Web / ERP / SaaS	Next.js/React, FastAPI/NestJS/Go, Postgres, Redis, K8s; ERPNext/Odoo as base	Playwright browser farm, ephemeral Postgres, k6 load, OWASP ZAP	Accountant sign-off on tax, payroll, GL; data-migration rehearsal
Mobile	Flutter or React Native (native Swift/Kotlin when needed)	Android emulators + real device shelf; Mac minis required for iOS builds ; Appium	Store policy review, signing keys held by client, privacy labels
IoT / embedded	C/Rust firmware (Zephyr/FreeRTOS/ESP-IDF), MQTT, cloud ingestion, OTA	QEMU/Renode for scale-out + hardware-in-the-loop rack for timing and drivers	Human firmware engineer for safety-critical/real-time; signed OTA + rollback mandatory
Network	IaC and config (Ansible, Terraform, NETCONF/gnMI), SDN controllers, monitoring, eBPF tools	Containerlab/GNS3 virtual labs, Batfish config analysis, physical switch/firewall pair	Changes to live networks only via change window + human approval + auto-rollback

Domain	Typical stack generated	Verification environment	Extra gates / cautions
Cyber-security products	Hardening baselines, detection rules (Sigma/YARA), SIEM pipelines, scanners, secure code fixes	Isolated cyber range with deliberately vulnerable targets; Semgrep, Trivy, ZAP, Nuclei	Authorised targets only, signed scope, kill-switch, offensive tooling never touches Zone 0/1

Cross-domain products (an ERP with a mobile app and IoT gateways) are decomposed by the Orchestrator into per-squad task DAGs that meet at shared typed contracts (API schemas, message schemas).

12. ERP worked example: build in waves

From "I need an ERP for my manufacturing plant" to a live system in roughly 15 weeks (pilot estimate; two humans in the loop part-time).

Wave 0 Discovery	Interview, process map, chart of accounts, roles, data migration plan	Wk 1-2
Wave 1 Foundation	Auth/RBAC, multi-tenant, audit log, master data, CI/CD	Wk 2-4
Wave 2 Core	Inventory, purchasing, sales, invoicing (ERPNext/Odoo modules + customisation)	Wk 4-8
Wave 3 Finance & HR	GL, AP/AR, tax rules, payroll: accountant sign-off mandatory	Wk 8-11 human gate
Wave 4 Integrate	Bank / tax / e-invoice APIs, reports, data import, mobile approvals	Wk 10-13
Wave 5 Harden + Launch	Load test, pen-test, UAT, training docs, canary go-live	Wk 13-15
Wave 6 Operate	Hypercare, SLOs, monthly releases, ticket-to-patch loop	Wk 16+

13. Hardware requirements

Three tiers. Do not buy Tier B/C before a rented-GPU pilot proves your task mix and throughput. Prices are 2026 planning ranges; GPU pricing moves fast.

Component	Tier A: Pilot (1-3 concurrent projects)	Tier B: Production (10-25 projects)	Tier C: Scale (50+ projects)
GPUs	2-4x 48GB workstation/data-centre GPUs (RTX 6000 Ada/Pro, L40S) or rent 8-GPU cloud node	1 node 8x80GB (H100/H200/MI300-class) + 1 node 4x48GB	3-6 nodes 8x H200/B200-class, NVLink, plus small-model pool
Reasoning capability	Mid-size models (32-72B) local; large MoE via INT4 on rented node	Large MoE at FP8 locally	Multiple replicas, dedicated pools per model
CPU / RAM (per GPU node)	32-64 cores, 256 GB	2x 64-core, 1-2 TB	2x 96-core, 2 TB
Sandbox / build farm	1x 64-core, 256-512 GB, 4 TB NVMe	3-4 nodes x 64-128 cores, 512 GB-1 TB	10+ nodes, autoscaled, some in cloud
Control plane + data	3 small VMs or 1 server	3-node K8s, HA Postgres, object store 50-100 TB	Multi-AZ, Ceph/MinIO 500 TB+
Storage	8-16 TB NVMe	50-100 TB NVMe + 200 TB HDD/object	Distributed NVMe/object, 1 PB+
Network	10 GbE	25/100 GbE fabric, redundant ToR switches	100-400 GbE / InfiniBand for GPU nodes
Power + cooling	1-2 kW, office/server room OK	~10-14 kW for GPU nodes: needs colocation or dedicated cooling	40-100+ kW, colo or cloud
Approx. capex	\$20-45k (or \$2-5k/month rented)	\$300-500k	\$1.5-4M (or hybrid with cloud burst)
Indicative capacity	~2-4 unattended build streams	~40-80 concurrent agent tasks	200-500+ tasks

Labs (add per domain, not day one)

Lab	Contents	When needed	Approx. cost
Mobile lab	2-4 Mac minis (M-series) for iOS/Xcode, 8-12 Android + 4-6 iOS devices, USB hub / device-farm software	Phase 4	\$8-20k
IoT HIL rack	Dev boards (ESP32, STM32, nRF52, RPi, Jetson), programmers/debuggers (J-Link), logic analyser, programmable PSU, relay boards, sensors, MQTT/LoRa gateways	Phase 5	\$6-20k
Network lab	Physical L3 switch + router + firewall pair, Wi-Fi APs, traffic generators; Containerlab for virtual scale	Phase 5	\$4-12k
Cyber range	Isolated servers (or VLAN) with vulnerable images, C2 simulation, packet capture	Phase 4	\$3-10k
Ops workstations	Bastion host, hardware keys (YubiKey), monitors for on-call	Phase 0	\$3-6k
DR / backup	Off-site encrypted backup target, UPS, second-site replica of control plane	Phase 3	\$5-25k

Rent first: a cloud 8xH100 node is roughly \$15-30/hour on demand (less reserved). Four weeks of rented benchmarking costs a small fraction of a purchase and tells you which models and how many GPUs you actually need.

14. Software requirements (stack by layer)

Layer	Recommended	Alternatives	Notes
OS / virtualisation	Ubuntu 22.04/24.04 LTS, KVM	RHEL/Rocky, Proxmox	NVIDIA driver + CUDA pinned per cluster
Container / orchestration	Kubernetes (k3s for pilot, RKE2/upstream for prod), containerd	Nomad, Docker Swarm (pilot only)	GPU operator, Kueue/Volcano for GPU scheduling
Model serving	vLLM, SGLang	TGI, llama.cpp (small/edge), TensorRT-LLM	Prefix cache, FP8/AWQ, speculative decoding
Constrained decoding	XGrammar / Outlines (via vLLM/SGLang)	Ilguidance, GBNF, BAML	See section 9
Workflow engine	Temporal	Argo Workflows, Prefect, Dagster	Durable, resumable, human-approval signals
Agent framework	Own thin orchestration + LangGraph / PydanticAI for role logic	OpenAI-Agents-style SDKs, CrewAI, AutoGen	Keep framework replaceable; state lives in Temporal
Tool protocol	MCP servers behind a policy broker	Custom RPC	Every tool has an argument schema + allowlist
Sandboxing	Firecracker microVMs, gVisor; Kata for K8s	Docker rootless, Podman	No default egress; per-tenant networks
Runtime security	Tetragon / Falco (eBPF), seccomp, AppArmor	Wazuh	Audit exec + network of sandboxes
Source control + CI	Forgejo or GitLab CE, Woodpecker/GitLab CI/Tekton	Gitea, GitHub Enterprise	One repo per client project, client is owner
Artifact/package mirrors	Harbor, Nexus/Artifactory, Verdaccio, devpi	Cloud registries	Air-gapped builds from mirror
Datastores	PostgreSQL + pgvector, Redis, MinIO	Qdrant/Weaviate, Ceph	Backups + PITR
Messaging	NATS or Kafka	RabbitMQ	Agent events, telemetry
Policy	Open Policy Agent (OPA)	Cedar, Kyverno	Gates, RBAC, egress rules
Identity / secrets	Keycloak (SSO), HashiCorp Vault / OpenBao, HSM for signing	Authentik, cloud KMS	Short-lived credentials
Observability	OpenTelemetry, Prometheus, Grafana, Loki, Tempo, Langfuse	ELK, SigNoz	Cost + trace per project
laC / GitOps	Terraform/OpenTofu, Ansible, Argo CD/Flux, Helm	Pulumi, Crossplane	Generated for client environments
Testing / quality	pytest, Jest/Vitest, Playwright, k6, Hypothesis, Stryker	Cypress, Locust	Test frameworks per target language

Layer	Recommended	Alternatives	Notes
Security scanning	Semgrep, Trivy, Gripe, Gitleaks, OWASP ZAP, Nuclei, Syft (SBOM), Cosign	SonarQube, Snyk (hosted)	SLSA provenance + signed releases
Domain tooling	Android SDK + emulators, Xcode (Mac), Flutter/RN; Zephyr/ESP-IDF, QEMU, Renode; Containerlab, Batfish, Scapy	GNS3, PlatformIO	Per squad images
Portal / admin UI	Next.js + TypeScript, Postgres, WebSockets/SSE	Any modern stack	Dogfood: build it with the platform later
ERP base	ERPNext (Frappe) or Odoo Community	Custom modular monolith	Check licence terms for SaaS resale (Odoo/ERPNext differ)

15. Maintenance and operations

- **Monitoring:** SLIs/SLOs per product (availability, latency, error rate, data freshness); alerts routed to the SRE agent, then human on-call by severity.
- **Ticket-to-patch loop:** support agent classifies and reproduces a ticket, writes a failing test, dev agent fixes, verifiers run, canary release. Bugs become regression tests permanently.
- **Routine upkeep:** weekly dependency + CVE updates as automated PRs, monthly release train, quarterly disaster-recovery restore drill, annual pen-test by an external human firm.
- **Incidents:** runbooks as code; auto-remediation only inside blast-radius limits (restart, scale, rollback); blameless postmortems generate new tests and runbook edits.
- **Platform self-maintenance:** nightly model/prompt eval, drift alarms, prompt and model versioning, canary of new model versions on shadow traffic before promotion.
- **Commercial model:** build fee (fixed scope) + monthly plan (hosting, monitoring, N change requests, SLA tiers).

16. Security, compliance and governance

Risk	Control
Prompt injection via client docs, web pages, dependency READMEs	Treat all external text as data; guard model + tool allowlist; no tool can exceed role scope; human gate on sensitive actions
Malicious or buggy generated code	Sandboxes with no egress, SAST/DAST, dependency allowlist, SBOM, reviewer agent, signed builds
Cross-tenant leakage	Per-tenant repos, namespaces, encryption keys, vector-store partitions; no shared long-lived context
Secrets exposure	Vault, short-lived tokens, Gitleaks in every PR, secrets never in prompts
Supply-chain attacks	Package mirror + pinning + provenance checks (SLSA), Cosign signing, licence scanner
Model theft / poisoning	Weights on isolated storage, hash-verified downloads, fine-tune data provenance
IP and licensing	Track model licences (some open weights restrict commercial use), OSS licence scan, client owns output contractually
Regulatory	Data residency by region, DPDP/GDPR processes, audit log, SOC 2 / ISO 27001 roadmap; sector rules (PCI, HIPAA) handled per-project with expert review

Risk	Control
Autonomous actions on live systems	Change windows, approvals, blast-radius caps, auto-rollback, global kill-switch

17. Team, cost and KPIs

Minimum team for the pilot (12-16 weeks)

Role	FTE	Focus
Platform / ML infra engineer	1	GPUs, vLLM/SGLang, routing, evals
Backend + workflow engineer	1	Temporal, control plane, artifact registry, billing
Agent / harness engineer	1-2	Prompts, tools, verifiers, sandbox integration
DevOps / security engineer	1	K8s, sandboxes, Vault, scanners, IaC
Frontend engineer (portal)	0.5-1	Idea portal, approvals, dashboards
Domain experts (ERP consultant, accountant)	0.2-0.5	Acceptance tests, sign-off, ERP templates
Product owner	0.5	Priorities, pilot client, pricing

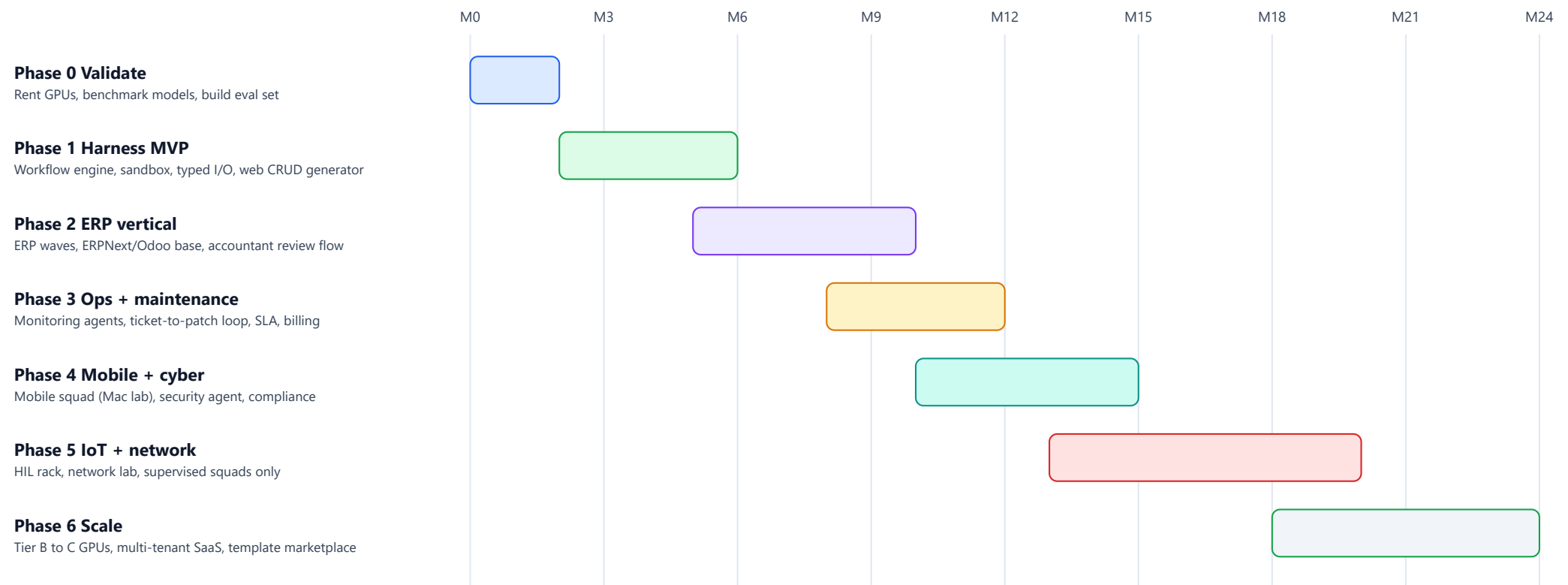
Indicative pilot budget: team cost + Tier A hardware or rented GPUs + labs deferred = roughly **\$250-600k** depending on geography. Tier B hardware is added only after the pilot passes exit criteria.

KPIs

Area	KPI	Pilot target
Quality	Golden-task unattended pass rate	>70% (Phase 1), >85% (Phase 3)
Quality	Escaped defects per release (P1/P2)	0 P1 in first 30 days
Speed	Idea to staging (web CRUD app)	<1 week; ERP core <8 weeks
Cost	Compute + human-review cost per delivered feature	Tracked per project, falling monthly
Safety	Sandbox escapes / secret leaks	0
Ops	MTTR for P2 incidents; SLO attainment	<4 h; 99.5%+
Business	Client NPS; gross margin per project	>40 NPS; >50% margin at scale

18. Roadmap (24 months)

Validate cheaply, ship one vertical (ERP) end-to-end, then widen domain by domain as evals justify.



Months from start; phases overlap. Exit criteria: Phase 1 = 70%+ of golden tasks pass unattended; Phase 2 = one pilot ERP live with zero P1 defects for 30 days.

19. Risks and mitigations

Risk	Likelihood / impact	Mitigation
Local models underperform frontier models on hard tasks	High / Medium	Router with per-task evals; opt-in hosted fallback; decompose tasks smaller; fine-tune on accepted PRs
Agents loop or burn compute	Medium / Medium	Attempt caps, budgets per task, early escalation, cost dashboards
ERP correctness in regulated logic	Medium / High	Open-source ERP core, accountant-authored acceptance tests, mandatory human gate
GPU capex wasted or obsolete	Medium / High	Rent first; buy in stages; keep inference behind an abstraction so nodes are swappable
Scope creep across too many domains	High / High	One vertical at a time; exit criteria per phase; domain squads only when the lab exists
Security incident from generated code	Low-Medium / High	Layered scanning, sandboxing, pen-tests, bug bounty, kill-switch
Client expectations of "fully automatic"	High / Medium	Transparent gates, staged deliveries, clear SLAs and change-request rules
Talent gap for platform team	Medium / Medium	Start with 4-5 strong generalists; use the platform to build the platform

20. First 30 days checklist

- **Week 1:** Decide product framing (Idea-as-a-Service), pick the ERP vertical and one pilot client/use-case; set up Git, CI, Vault, observability skeleton; rent an 8-GPU node.
- **Week 1-2:** Write 40-60 golden tasks (spec + tests) for web/ERP; benchmark 3-5 open models on them via vLLM with constrained decoding; record pass rate, latency, cost.
- **Week 2:** Build the sandbox runner (Firecracker or gVisor) with no-egress policy and a package mirror; run 100 tasks through it.
- **Week 2-3:** Stand up Temporal; implement the single-task harness loop (diagram 4) end-to-end for one task type (CRUD endpoint + tests).
- **Week 3:** Implement Product + Architect agents with typed schemas (PRD, OpenAPI, DDL); build the approval inbox skeleton.
- **Week 4:** Run the first end-to-end "idea to staging" on a small app at autonomy L0; review every trace; convert failures into new golden tasks.
- **Week 4:** Decide with data: model mix, GPU tier to buy, which gates to keep manual; publish the Phase 1 plan and budget.

Companion files in this folder: `Hardware_Software_BOM.csv` (sizing table), editable `plan.html`, and `diagrams.mjs` / `build.mjs` to regenerate the PDF after edits.